

Программирование на Bash: Базовое программирование на Bash. Коды возврата, функции.



Андрей
Тряпичников



Андрей Тряпичников

Системный администратор

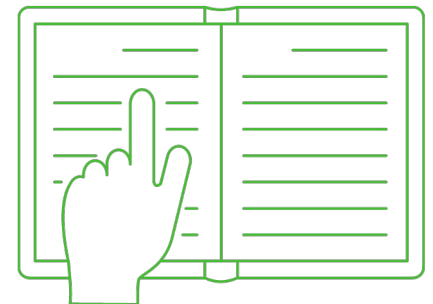
Одноклассники

Предисловие

На этом занятии мы поговорим о:

- основах программирования на Bash;
- кодах возврата;
- функциях, условиях и циклах в языке Bash.

По итогу занятия вы получите представление о работе Bash и научитесь программировать на Bash.





План занятия

1. [Предисловие](#)
2. [Основы Bash-скриптинга](#)
3. [Переменные и коды ответов](#)
4. [Условные операторы](#)
5. [Циклы](#)
6. [Функции](#)
7. [Итоги](#)
8. [Домашнее задание](#)



Основы Bash-скриптинга

Обзор

Bash (Bourne-again shell) – самая популярная оболочка командной строки для Linux.

Помимо внешних утилит имеет широкий набор встроенных функций, таких как циклы, условия и прочее.

Простейший скрипт - последовательный набор команд для выполнения.

Пример простейшего скрипта:

```
#!/bin/bash
```

```
echo "Creating file"
```

```
touch file
```

```
echo "Done"
```

Запуск скриптов

Для запуска скрипта пользователь должен иметь права на его чтение и выполнение:

```
chmod +rx filename.sh
```

В начале скрипта необходимо указать интерпретатор:

```
#!/usr/bin/env bash
```

Если этого не сделать, скрипт будет выполнен с `/bin/sh`

Если прав на выполнение нет то можно просто запустить:

```
bash scriptname
```

`/bin/sh` далеко не всегда ссылается на `bash`. В некоторых дистрибутивах это более примитивный шелл.



Переменные и коды ответов

Переменные

Присвоение значений
переменным

Простое:

```
a=2
```

С помощью let (в
арифметическом контексте):

```
let a=2
```

```
((a=2))
```

Чтение ввода пользователя:

```
read a
```

Переменные

Присвоение значений
переменным

Простое:

```
a=2+2
```

С помощью let (в
арифметическом контексте):

```
let a=2+2
```

```
((a=2+2))
```

Чтение ввода пользователя:

```
read a
```

Подстановка значений переменных

```
echo "Hello $name"
```

```
echo "Hello ${name}s"
```

```
echo "Hello ${name:-Andrey}"
```

Подстановка вывода команды

```
echo "Today is $(date +%Y-%m-%d)"
```

```
echo "Today is `date +%Y-%m-%d`"
```

Подстановка результата
арифметической операции

```
echo "2 + 2 = $((2+2))"
```

Перенаправление потоков

Стандартные потоки ввода/вывода:

- **0** (STDIN) ввод пользователя;
- **1** (STDOUT) стандартный поток вывода;
- **2** (STDERR) стандартный поток ошибок.

Для перенаправления результата выполнения команды в другую команду используется **|** (pipe).

Пример:

```
cat /etc/passwd | wc -l
```

wc -l – используется для подсчета количества строк.

Еще пример:

```
ps aux | grep bash | wc -l
```

Коды ответов

Код ответа (Return code, код возврата) – код который возвращает команда или функция после выполнения.

Код возврата предыдущей команды хранится в переменной **\$?**

Пример:

```
cat /etc/not_existing_file
```

```
echo $?
```

Код возврата – число от 0 до 255

Традиционно:

- 0 – успех;
- Любое другое число – как правило, код ошибки (свой у каждого приложения).



Экранирование, пробел и *

Пробел

Пробел в bash интерпретируется по умолчанию, как разделитель полей, поэтому там, где поля разделять не нужно, он должен быть заэкранирован:

```
myvar=my\ value           myvar='my value'
```

```
set -- $myvar
```

```
echo $1
```

При подстановке экранирование тоже может быть нужно:

```
set -- "$myvar"
```

```
echo "$1"
```

Кавычки (экранирование)

Одинарные кавычки означают буквальную подстановку всего, что находится внутри. Подстановка переменных и команд не осуществляется:

```
echo '$myvar $(date)'
```

Двойные кавычки экранируют почти всё, кроме подстановки:

```
echo "$myvar $(date)"
```

wildcard и другие символы

Другие символы могут тоже нуждаться в экранировании, например ? или *:

```
echo * ?
```

* также известен как wildcard или asterisk

По умолчанию “разворачивается” в список всех имён файлов в указанной директории:

```
echo *
```

```
echo /etc/*
```

? означает один любой символ:

```
echo /????
```




Условные операторы

Условные операторы

В Bash реализованы 2
условных оператора: **if** и
case.

- **if** используется для проверки кода ответа команды
- **case** для выбора последовательности действий для одного из множества вариантов значения переменной;

Синтаксис **if**
(**else** и **elif** опциональны!):

```
if cmd; then
    ...
elif other_cmd; then
    ...
elif third_cmd
then
    ...
else
    ...
fi
```

Условные операторы

В Bash реализованы 2 условных оператора: **if** и **case**.

- **if** используется для проверки кода ответа команды
- **case** для выбора последовательности действий для одного из множества вариантов значения переменной;

Синтаксис case:

```
case "$variable" in
    "foo" )
        cmd1
        cmd2
        ...
    ;;
    "bar" ) cmd3; cmd4; ... ;;
    * ) cmd5 ;;
esac
```

Проверка условий

2 варианта написания:

`[[ ]]` или `[ ]` (и в их поведении возможны различия!)

Примеры:

`[[ $a -eq 8 ]]`

`[ -d /tmp ]`

Инверсия результата достигается с помощью «!».

Например:

`[[ ! -d /tmp ]]` будет успешна если директория не существует

Таблица проверочных выражений

Работа с файлами	
-e – Проверить существует ли файл или директория -f – Существует и является файлом -d – Существует и является каталогом -h – Существует и является символической ссылкой -s – Файл существует и он не пустой -r – Файл существует и доступен для чтения -w – ... для записи -x – ... для выполнения	
Работа со строками	Операции с числами
-z – Пустая строка -n – Не Пустая строка == – Равно != – Не равно	-eq – Равно -ne – Не равно -lt – Меньше -le – Меньше или равно -gt – Больше -ge – Больше или равно

Пример if и case

```
if [[ $port == '80' || $port == '8080' ]];  
then  
    echo "This is HTTP"  
elif [[ $port == '22' ]]; then  
    echo "This is SSH"  
elif [[ $port == '53' ]]; then  
    echo "And this is DNS"  
else  
    echo "I dont know what is that"  
fi;
```

```
case "$port" in  
    ('80'|'8080')  
        echo "This is HTTP"  
        ;;  
    '22')  
        echo "This is SSH"  
        ;;  
    '53')  
        echo "And this is DNS"  
        *)  
        echo "I dont know what is that"  
        ;;  
esac
```

Логические операторы

Для упрощения записи команд есть 3 оператора, позволяющие избежать ненужного усложнения с помощью `if` или `case`:

- `команда1 ; команда2` – команда2 выполняется после команды1 независимо от результата ее работы ;
- `команда1 && команда2` – команда2 выполняется только после успешного выполнения команды1 (т.е. с кодом завершения 0), это аналог операции **AND** (Логическое И);
- `команда1 || команда2` – команда2 выполняется только после неудачного выполнения команды1 (т.е. код завершения команды1 будет отличным от 0), это аналог операции **OR** (Логическое ИЛИ).



Арифметические операции

Строки и числа

Любая переменная по умолчанию считается строкой. Даже если в неё записано число. Поэтому операции с числами по умолчанию не осуществляются:

```
a=2+2
```

```
echo $a
```

Однако, bash поддерживает целочисленную арифметику, для этого используется специальный синтаксис:

```
((a=2+2))
```

```
echo $a
```

Инкременты и присвоение значений

Как и другие языки программирования, bash предоставляет сокращённый синтаксис для инкрементации чисел и присвоения нового значения числу после осуществления математической операции с ним:

```
a=1
```

```
((a++))
```

```
((a--))
```

```
((a*=3))
```

Подстановка

Иногда нам нужно подставить результат выражения в текст. В этом случае можно использовать подстановку результата арифметического выражения:

```
myresult=$((100 + 46))
```

```
echo $myresult
```

При подстановке переменных внутри арифметических операций обязательно использовать \$, любой текст внутри двойных круглых скобок интерпретируется как имя переменной:

```
a=2
```

```
echo $((a*a))
```

```
echo $(($a*$a))
```

Сравнение

Среди арифметических операций поддерживается сравнение:

`((a < 2))`

`((a != 0))`

При этом сравнение с нулём часто записывают так:

`((a))`

Эта проверка возвращает 0, если число не равно нулю и 1, если равно. Что часто используется в `if`.

Обратите внимание, что наличие или отсутствие пробелов после открывающими и перед закрывающими скобками иррелевантно, в отличие от команд `[[`, `[` и `test`.



Циклы

Циклы: while

while – самый простой цикл

Синтаксис:

```
while cmd do
```

```
    echo "Ok"
```

```
done
```

➡ здесь cmd - это проверка, код её выхода определяет, продолжит ли работать цикл.

➡ Бесконечное выполнение команд пока условие выполняется.

Циклы: while

while – самый простой цикл

Синтаксис:

```
while cmd do
```

```
    echo "Ok"
```

```
done
```

➡ здесь cmd - это проверка, код её выхода определяет, продолжит ли работать цикл.

➡ Бесконечное выполнение команд пока условие выполняется.

Пример:

```
i=0
```

```
while [[ $i -le 4 ]]; do
```

```
    echo $i
```

```
    ((i++))
```

```
done
```

Циклы: while

while – самый простой цикл

Синтаксис:

```
while cmd do
```

```
    echo "Ok"
```

```
done
```

➡ здесь cmd - это проверка, код её выхода определяет, продолжит ли работать цикл.

➡ Бесконечное выполнение команд пока условие выполняется.

Пример:

```
i=0
```

```
while [[ $i -le 4 ]]; do
```

```
    echo $i
```

```
    ((i++))
```

```
done
```

Бесконечный цикл:

```
while true; do
```

```
    echo 'I can do that all day'
```

```
done
```


Циклы: for

Если нет условия, а есть просто список элементов – используем **for**

```
for имя in элемент1 элемент2; do
    ...
done
```

➡ Выполняет тело цикла для каждого из **элементов**, загружая их поочередно в переменную **имя**

Пример:

```
#!/bin/bash

for file in /tmp/*; do
    echo "$file"
done
```

Циклы: for

Если нет условия, а есть просто список элементов – используем **for**

```
for имя in элемент1 элемент2; do
    ...
done
```

➡ Выполняет тело цикла для каждого из **элементов**, загружая их поочередно в переменную **имя**

Пример:

```
#!/bin/bash
for file in /tmp/*; do
    echo "$file"
done
```

Еще пример:

```
#!/bin/bash
for $item in coffee tea; do
    echo "We have $item"
done
```



Функции

Функции

Функция – часть кода, вынесенная отдельно, для многократного использования.

Объявление функции в Bash:

```
myfunc () {  
    cmd1  
    cmd2  
    if ...  
}
```

Вызов функции:

```
myfunc
```

Функции

Функция – часть кода, вынесенная отдельно, для многократного использования.

Объявление функции в Bash:

```
myfunc () {  
    cmd1  
    cmd2  
    if ...  
}
```

Вызов функции:

```
myfunc
```

Пример:

```
#!/bin/bash  
  
currentdate () {  
    echo "Current time:"  
    date  
}  
  
currentdate
```

Функции: параметры

Для многократного использования функцию обычно требуется запускать с разными параметрами.

Параметры – доступные только внутри тела функции переменные.

Например, вызов функции:

`function 5 2`

Имеет 2 параметра, к которым можно обратиться внутри самой функции с помощью переменных

`$1` и `$2`

Функции: параметры

Для многократного использования функцию обычно требуется запускать с разными параметрами.

Параметры – доступные только внутри тела функции переменные.

Например, вызов функции:

```
function 5 2
```

Имеет 2 параметра, к которым можно обратиться внутри самой функции с помощью переменных `$1` и `$2`

Пример функции с параметром:

```
#!/bin/bash
```

```
division () {
```

```
    if [[ $2 -ne 0 ]] then
```

```
        echo "$1/$2 = $((($1/$2))"
```

```
    else
```

```
        echo "division by zero"
```

```
    fi
```

```
}
```

```
division 15 5
```

```
division 4 2
```

```
division 3 0
```

Функции: код возврата

Указать код возврата для скрипта можно с помощью команды `exit`.

Выход из функции осуществляется с помощью `return`.

➡ Это позволяет передавать ошибку и обрабатывать, если уже известно, что что-то пошло не так.

➡ По умолчанию функции и скрипты выходят с кодом выхода последней команды в них.

Функции: код возврата

Указать код возврата для скрипта можно с помощью команды `exit`.

Выход из функции осуществляется с помощью `return`.

➡ Это позволяет передавать ошибку и обрабатывать, если уже известно, что что-то пошло не так.

➡ По умолчанию функции и скрипты выходят с кодом выхода последней команды в них.

Пример функции с параметром:

```
#!/bin/bash

division () {
    if [[ $2 -ne 0 ]]; then
        echo "$1/$2 = $((($1/$2))"
        return 0
    else
        echo "division by zero"
        return 1
    fi
}

division 15 5
division 4 2
division 3 0
```

Отладка

Интерпретатору `#!/bin/bash` можно передавать параметры работы с помощью встроенной команды `set` в самом скрипте или при вызове интерпретатора, несколько полезных опций:

- `-n` – читать все команды, но не выполнять их;
- `-v` – выводить все строки по мере их обработки интерпретатором;
- `-x` – выводить все команды и их аргументы по мере их выполнения.



Итоги

Итоги

Сегодня мы рассмотрели базовые навыки программирования Bash и теперь:

- умеем составлять скрипты из команд;
- можем использовать функции, циклы и условия.



**Задавайте вопросы и
пишите отзыв о лекции!**

Андрей Тряпичников

a.tryapichnikov@gmail.com